E-commerce Application — Test Scenarios Analysis

Application: SauceDemo (https://www.saucedemo.com/)

1. Introduction

SauceDemo is a demo e-commerce application that allows users to log in, browse products, add items to a shopping cart, and complete the checkout process. Testing these features is important to ensure that the application works correctly, validates user input, and provides a reliable shopping experience.

2. Test Scenarios and Classification

No.	Test Scenario	Classification	Justification
1	Verify that the login validation logic rejects invalid credentials.	Unit Testing	Tests authentication-related logic independently using controlled inputs and mocked dependencies.
2	Verify that the product filtering logic returns products matching the selected category or search criteria.	Unit Testing	opening the browser.Checks the filtering logic in isolation without
3	Verify that the cart subtotal is calculated correctly based on product prices and quantities.	Unit Testing	Tests the calculation logic with known input values and expected results.
4	Verify that the login API accepts valid credentials and rejects invalid credentials with appropriate responses.	API Testing	Tests the authentication endpoint, response status, and response body directly.
5	Verify that the products API returns the expected product details and prices.	API Testing	Checks the service response and data structure without interacting with the user interface.
6	Verify that the cart API correctly updates item quantities and cart contents.	API Testing	Tests server-side cart operations and verifies the returned data.
7	Verify that the checkout process rejects requests with missing required customer information.	API Testing	Validates server-side input validation and error handling at the API boundary.
8	Verify that the payment service handles successful and declined payment requests correctly.	API Testing	Tests payment responses and transaction states using a controlled test environment or payment sandbox.
9	Verify that a user can log in through the login page and access the products page.	UI Testing	Tests the visible login form, user interactions, and navigation to the products page.
10	Verify that a user can select a product, add it to the cart, and see the correct item displayed.	UI Testing	Checks the interaction between product controls, the cart icon, and the cart page.
11	Verify that a user can enter checkout information and complete the checkout flow.	UI Testing	Tests the user journey across the cart, checkout information, overview, and confirmation pages.
12	Verify that the application displays an appropriate message when checkout information is missing or invalid.	UI Testing	Checks visible validation messages and confirms that the user can understand and correct the error.

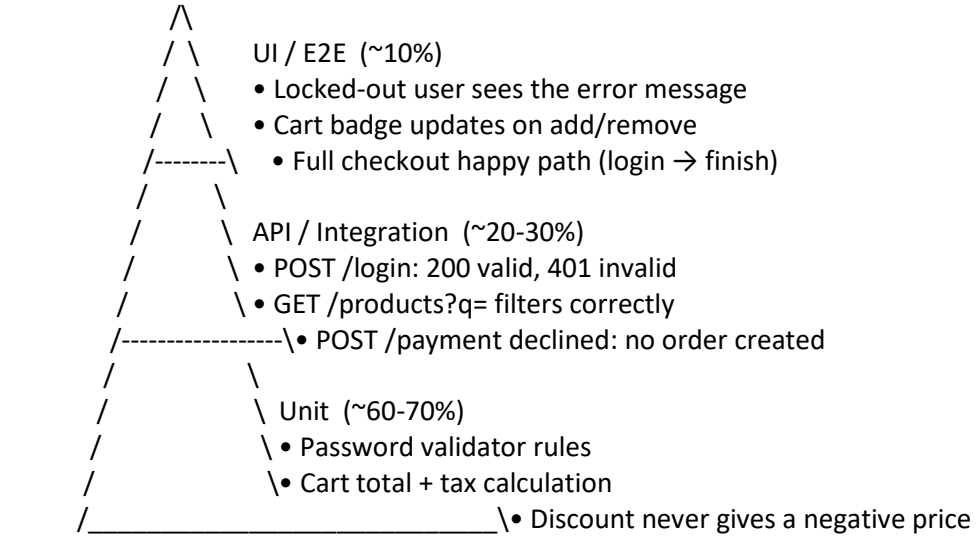
3. Explanation of the Testing Levels

Unit Testing: Unit tests focus on small pieces of code, such as validation rules, product filtering, and price calculations. They are generally fast and help developers identify defects early.
API Testing: API tests verify communication between application components. They check requests, responses, validation rules, and data integrity without relying on the browser interface.
UI Testing: UI tests verify the application from the user's perspective. They cover visible elements, navigation, error messages, and complete shopping journeys. UI tests are usually slower and more sensitive to interface changes.

4. Conclusion

A reliable e-commerce testing strategy combines unit, API, and UI testing. Unit tests provide fast feedback on individual business rules, API tests validate service behavior, and UI tests confirm that critical customer journeys work correctly. Using all three levels improves test coverage and helps identify defects at the appropriate level.

2. Test Pyramid



The Unit layer has the most tests, for four reasons:

- **Speed:** thousands run in seconds, so developers can run them on every commit.
- **Cost:** they're cheap to write and maintain, with no environment, data, or browser setup.
- **Stability:** they have few dependencies, so they rarely fail for random reasons (unlike flaky UI tests).
- **Precision:** a failure points to the exact function, while a failed E2E test could be caused by anything.

Going up the pyramid, each test costs more, runs slower, and breaks more easily, so you write fewer of them. An "ice cream cone" (mostly UI tests) gives you slow feedback, flaky pipelines, and expensive maintenance.